

How to Use perf to Find the One Line Killing Your C++ Performance

A 5-minute guide to `perf` that doesn't make you want to quit engineering.

5 min read · 6 days ago



Sagar

Following ▾

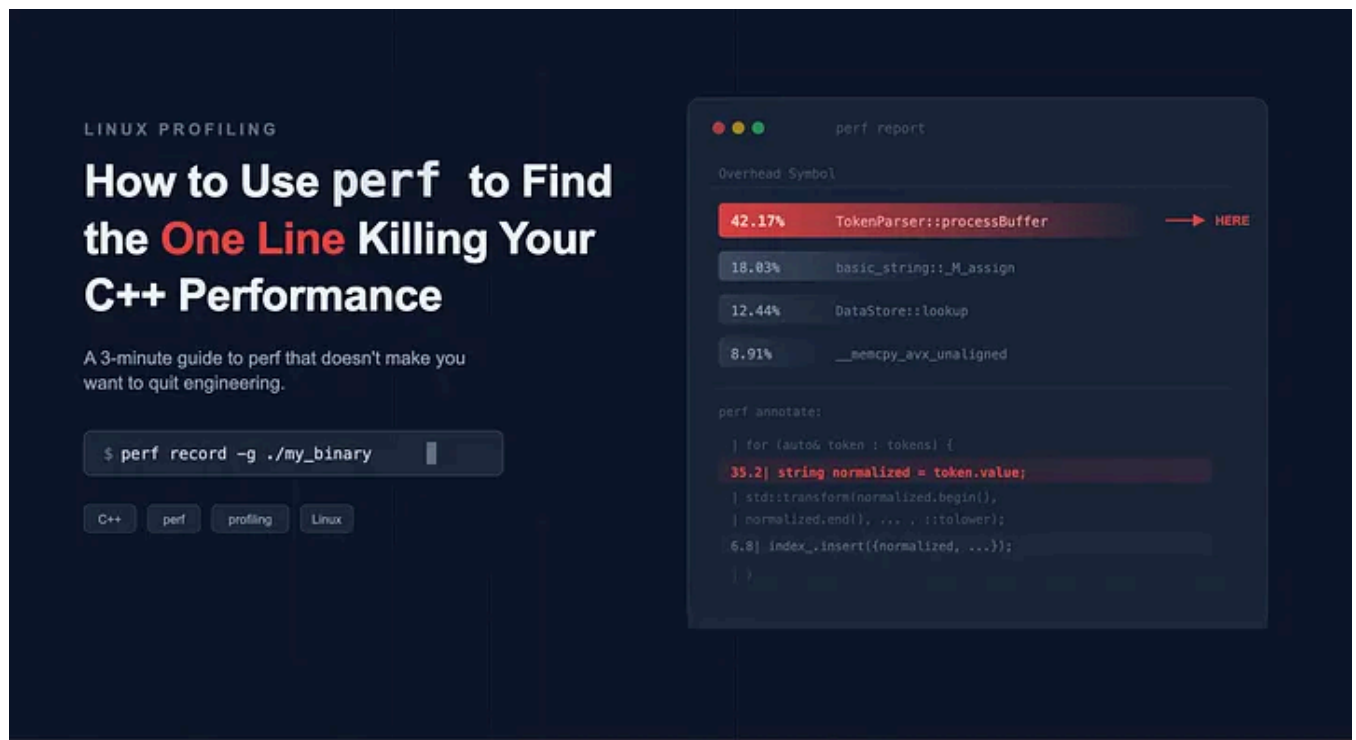


Listen



Share

... More



You've written C++. It compiles. It runs. It even produces correct results. But it's slow, and you're sitting there staring at 40,000 lines of code like a detective in a crime drama, except the murder weapon is a `std::map` and nobody's talking.

Let me introduce you to your new best friend: **perf**.

Step 0: Stop Guessing

I know what you're thinking. "It's probably that nested loop in the parser." No. It's never what you think it is. I once spent two days optimizing an algorithm that accounted for 3% of runtime while a `std::string` copy in a logging function was eating 40% of my CPU. My ego has never recovered.

Rule #1:

Measure. Don't guess. Your intuition is often wrong in performance work.

How to install perf

```
# Debian / Ubuntu
sudo apt install linux-tools-common linux-tools-generic linux-tools-$(uname -r)

# Fedora
sudo dnf install perf

# Arch
sudo pacman -S perf
```

[Open in app](#) ↗

≡ Medium



Step 1: Record a Profile (10 Seconds of Work)

Build your binary with debug symbols. You don't need to turn off optimizations — in fact, don't. You want to profile the *real* build.

```
# Compile with optimizations AND debug symbols. Yes, you can have both.
g++ -O2 -g -o my_beast main.cpp

# Record a profile. That's it. That's the whole command.
```

```
perf record -g ./my_beast
```

If your call stacks look wrong (missing frames, weird jumps) with -O2, try

```
perf record --call-graph dwarf ./my_beast
```

* Frame-pointer omission can break stack unwinding.
DWARF fixes it (at a small overhead cost).

The `-g` flag captures call graphs so you can see *who called what*. Without it, you're reading a phone book with no names.

This generates a `perf.data` file. Think of it as a crime scene photo album.

Step 2: Find the Hotspot (The Suspect Lineup)

```
perf report
```

You'll see something like this:

Overhead	Command	Shared Object	Symbol
42.17%	my_beast	my_beast	[.] TokenParser::processBuffer
18.03%	my_beast	libstdc++.so	[.] std::__cxx11::basic_string::_M_assign
12.44%	my_beast	my_beast	[.] DataStore::lookup
8.91%	my_beast	libc.so	[.] __memcpy_avx_unaligned

Look at that. `processBuffer` is hogging 42% of your CPU. And `_M_assign`? That's string copies. Eighteen percent of your life, spent copying strings. Beautiful.

Pro tip: Start with anything above ~5%. That's usually where the big wins are. You're looking for the big fish, not chasing minnows. Then come back for the smaller stuff if needed.

Step 3: Zoom In to the Exact Line (The Murder Weapon)

Here's where it gets good. You found the function. Now find the line.

```
perf annotate TokenParser::processBuffer
```

This shows you assembly interleaved with your source code (thanks to those `-g` debug symbols), with percentages next to each instruction:

```
35.2|         void TokenParser::processBuffer(const std::vector<Token>& tokens) {  
    |             for (auto& token : tokens) {  
    |                 std::string normalized = token.value;           // <--- look her  
    |                 std::transform(normalized.begin(),  
    |                             normalized.end(),  
    |                             normalized.begin(),  
    |                             ::tolower);  
    |                 index_.insert({normalized, token.position}); // <--- and here  
    |             }  
    |     }
```

There it is. Line by line, percentage by percentage. 35% of your time is spent copying `token.value` into a new `std::string` every single iteration. The `insert` with another copy is the accomplice.

You didn't need a profiling PhD. You needed two commands.



The Fix (Because I'm Not a Monster):

```
void TokenParser::processBuffer(const std::vector<Token>& tokens) {  
    std::string normalized; // reuse the buffer across iterations  
    for (auto& token : tokens) {  
        normalized = token.value;  
        std::transform(normalized.begin(),  
                        normalized.end(),  
                        normalized.begin(),
```

```
                ::tolower);  
    index_.emplace(std::move(normalized), token.position); // move, don't  
    }  
}
```

Moving the string outside the loop lets the capacity grow once and be reused, avoiding repeated allocations. **emplace** + **std::move** eliminates the second copy entirely—turning two heap-touching operations into zero. This kind of thing typically cuts 30-50% off hot loops like this.

Now the Real Fun Begins

Here's the thing — what we just did? That's **perf** on easy mode. We found *where* the CPU is spending time. That's step one. The "who done it."

But sometimes the *where* isn't enough. Your function is hot, you've eliminated obvious copies, there are no dumb allocations, and it's **still slow**. That's when you stop asking "where" and start asking "why."

perf is an absolute beast under the hood. It can tap into hardware performance counters — the stuff your CPU *actually tracks at the silicon level* — and tell you things like:

```
# Are you thrashing the cache?  
perf stat -e cache-misses,cache-references ./my_beast  
  
# Is the branch predictor crying?  
perf stat -e branch-misses,branches ./my_beast  
  
# The full picture — instructions, cycles, cache, branches, all of it  
perf stat -e cycles,instructions,cache-misses,branch-misses,L1-dcache-load-misses
```

You can even record cache misses as events and then annotate *which lines* are causing them:

```
perf record -e cache-misses -g ./my_beast
perf report  # now sorted by cache misses, not CPU time
```

Suddenly you're not guessing why **DataStore::lookup** is slow — you can *see* that it's missing L1 cache on every access because your data structure has the memory layout of a Jackson Pollock painting.

TLB misses. Page faults. NUMA effects. Instructions per cycle. **perf** can measure all of it. It's like having an X-ray machine for your CPU, and most engineers never look past the waiting room.

But don't start there. Seriously. Find the **hotspot first**. Fix the obvious stuff — the copies, the allocations, the algorithmic blunders. *Then* bring out the hardware counters when you've hit the wall and need to understand what the silicon is actually doing.

I have used `perf` for benchmarking/profiling in many of my tutorials on low latency C++ topics. Here is one such reference for you: [Branch Prediction Benchmark](#).

The Cheat Sheet — Minimal

WHAT YOU WANT	COMMAND
Record a profile	<code>`perf record -g ./my_binary`</code>
See which functions are hot	<code>`perf report`</code>
See which <i>lines</i> are hot	<code>`perf annotate SomeClass::someMethod`</code>
Profile a running process	<code>`perf record -g -p <PID>`</code>
See a quick summary without the TUI	<code>`perf report --stdio`</code>
Count cache misses	<code>`perf stat -e cache-misses ./my_binary`</code>
Count branch mispredictions	<code>`perf stat -e branch-misses ./my_binary`</code>
Profile by cache misses instead of CPU	<code>`perf record -e cache-misses -g ./my_binary`</code>

The perf Toolbox (What You Actually Use Daily)

Once you move beyond basic profiling, you'll find that most real-world debugging with **perf** relies on a small, reliable set of commands rather than the full feature set.

This is the practical toolkit you'll actually reach for in production.

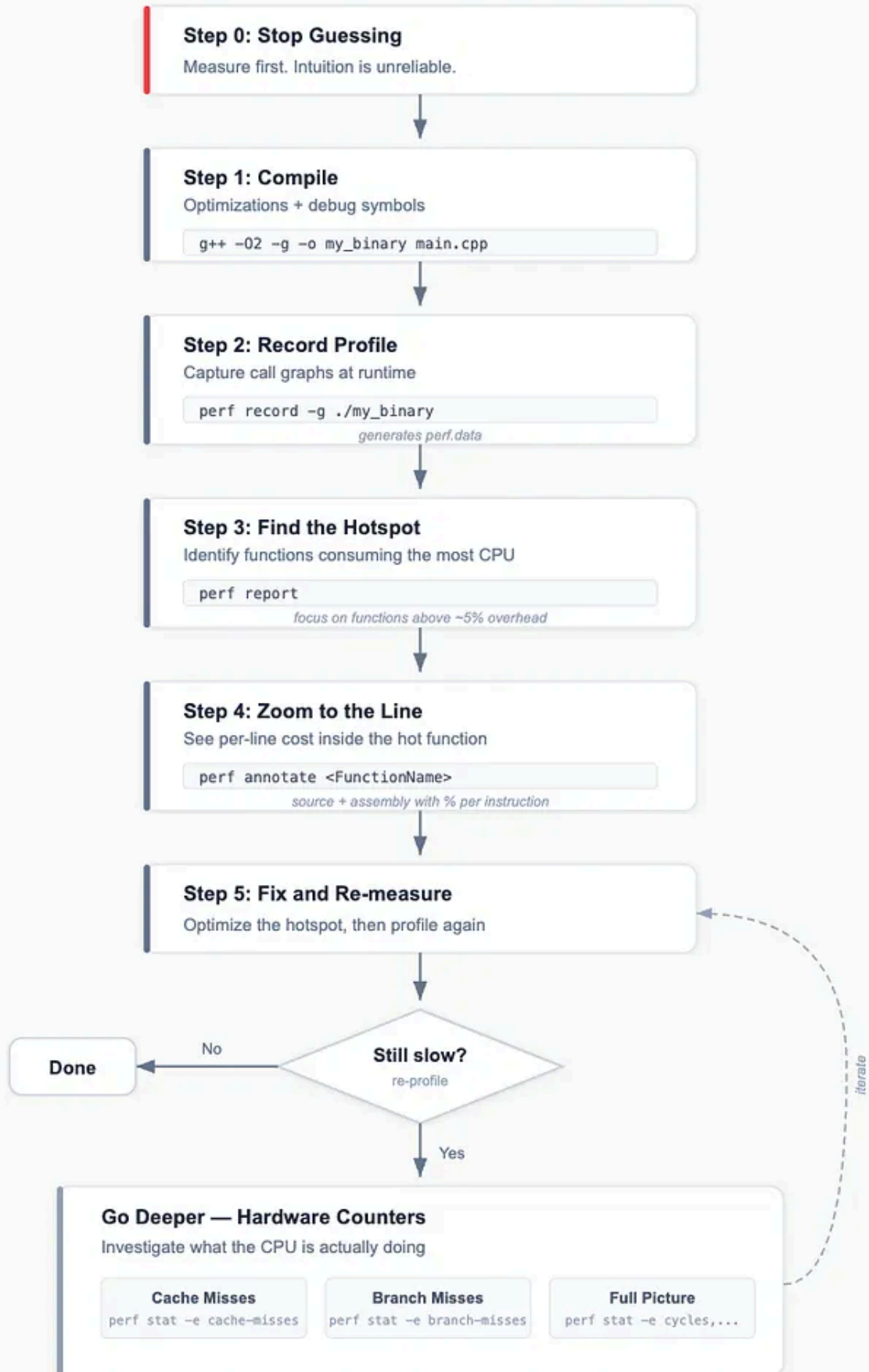
Use Case	Command	What It Does
Live CPU hotspots	<code>perf top</code>	Shows real-time functions consuming CPU right now
General profiling report	<code>perf report --sort=comm,dso,symbol</code>	Breaks down CPU usage by function, binary, and symbol
Sort by CPU cycles	<code>perf report --sort=cycles</code>	Finds most expensive code paths by execution cost
Sort by cache behavior	<code>perf report --sort=cache-misses</code>	Highlights memory/cache inefficiencies
Profile running process	<code>perf record -p <pid></code>	Attaches perf to an already running program
Attach via process name	<code>perf record -p \$(pidof my_beast)</code>	Quick attach without manually finding PID
Profile specific thread	<code>perf record -t <tid></code>	Focus only on a single thread (useful for thread pools)
Fix broken call stacks	<code>perf record --call-graph fp ./my_beast</code>	Fast call graph using frame pointers
Accurate call stacks	<code>perf record --call-graph dwarf ./my_beast</code>	More precise stacks, slightly higher overhead
Reduce noise in output	<code>perf report --no-children</code>	Shows only self-cost per function
Drill into symbols	<code>perf report -s symbol</code>	Focuses view on function-level hotspots
Flamegraph workflow	(external tools)	Visual full-call-stack CPU breakdown

Final Thoughts

Here is the big picture:

perf Profiling Workflow

From guesswork to data-driven optimization



Compile → Record → Report → Annotate → Fix → Repeat